

From greenfield to brownfield

Using agents in greenfield and brownfield projects



Take a CSV export feature. In a new application, you can decide which columns to include and agree on their order. In an existing application, even when adding a column, you need to clarify who already depends on the file as it is.

Where agents add value in brownfield projects

Application discovery and overview

Understand what the application does, its main features, components, technologies, and dependencies.

Architecture discovery

Reconstruct the existing architecture, including layers, services, integrations, data flows, and dependencies.

Business rules and domain discovery

Identify business logic embedded in code, validation, database queries, and application workflows.

Dependency and change-impact analysis

Understand what could be affected before making a change.

Security and technical-debt discovery

Use review agents to inspect code for potential vulnerabilities, outdated patterns, architectural violations, and maintainability concerns.

Test coverage and existing behavior discovery

Understand how the application currently behaves, which behaviors are protected by tests, and where important coverage is missing.

Discovery before development

Where agents add value in brownfield projects

- Application discovery and overview
- Architecture discovery
- Business rules and domain discovery
- Dependency and change-impact analysis
- Security and technical-debt discovery
- Test coverage and existing behavior discovery

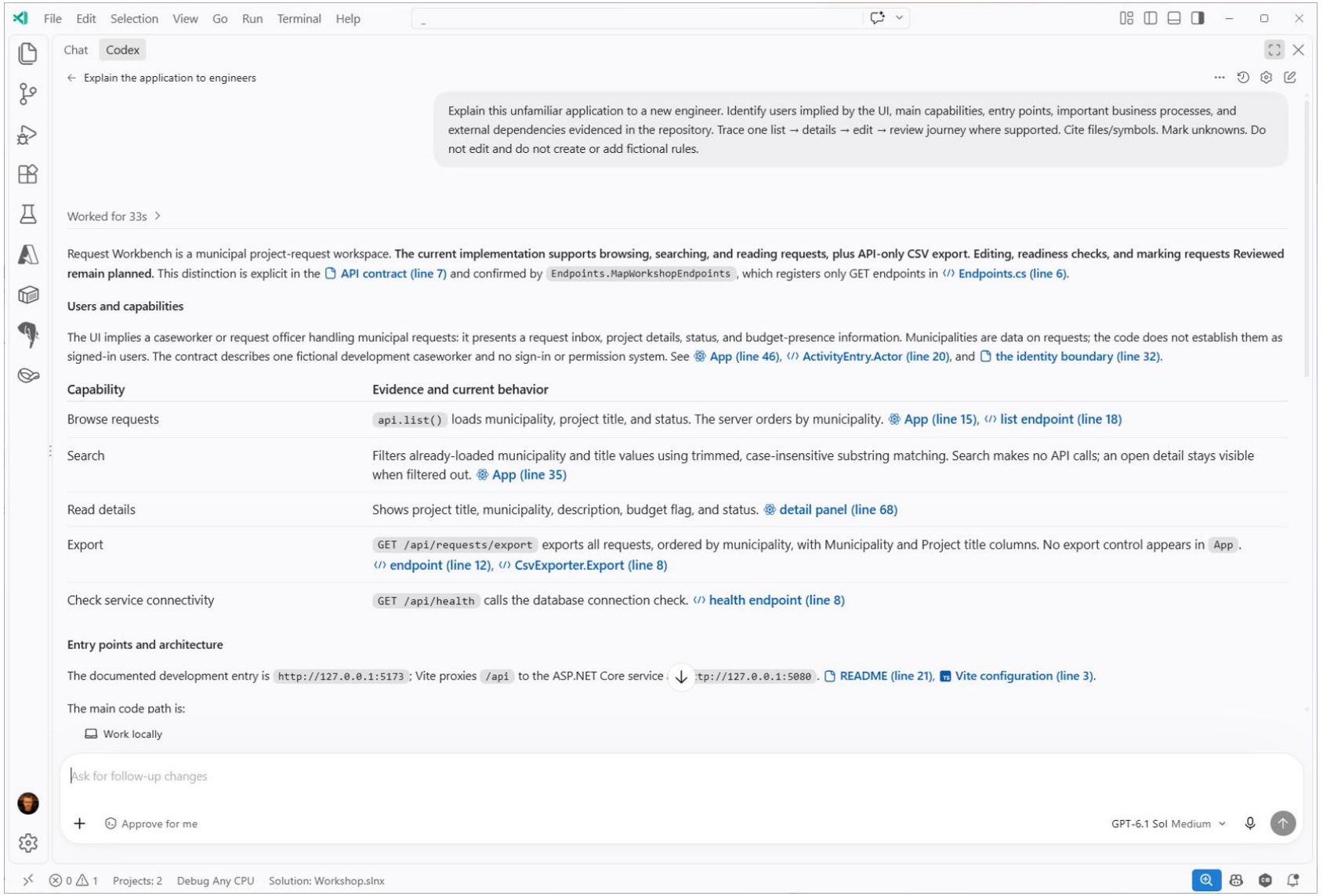
Instead of spending the first several days manually exploring the codebase, you could ask Codex to

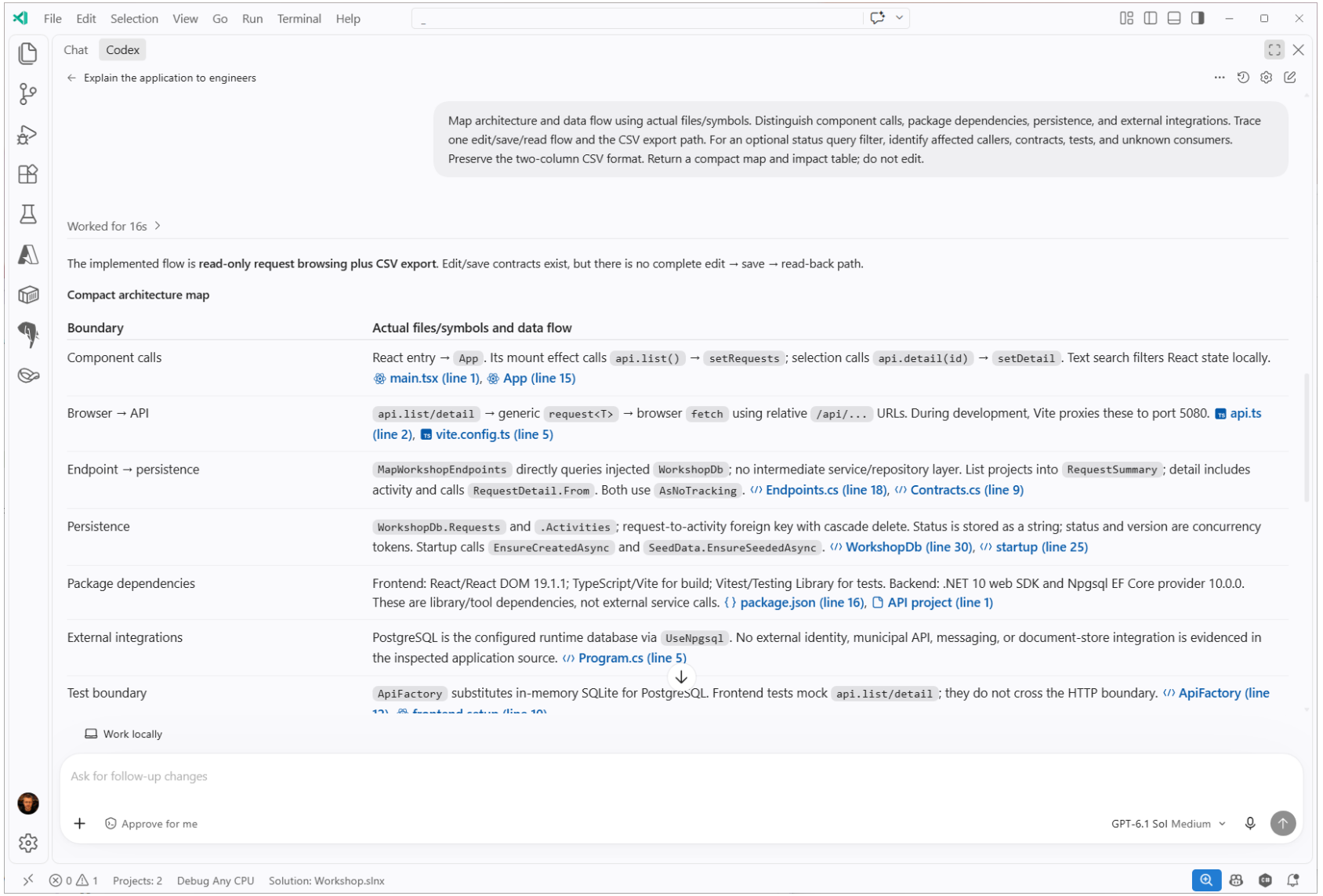
- Explain what the application does.
- Map its architecture and dependencies.
- Identify important business processes and rules.
- Discover how data flows through the system.
- Evaluate the existing test coverage.
- Identify potential security and maintainability risks.

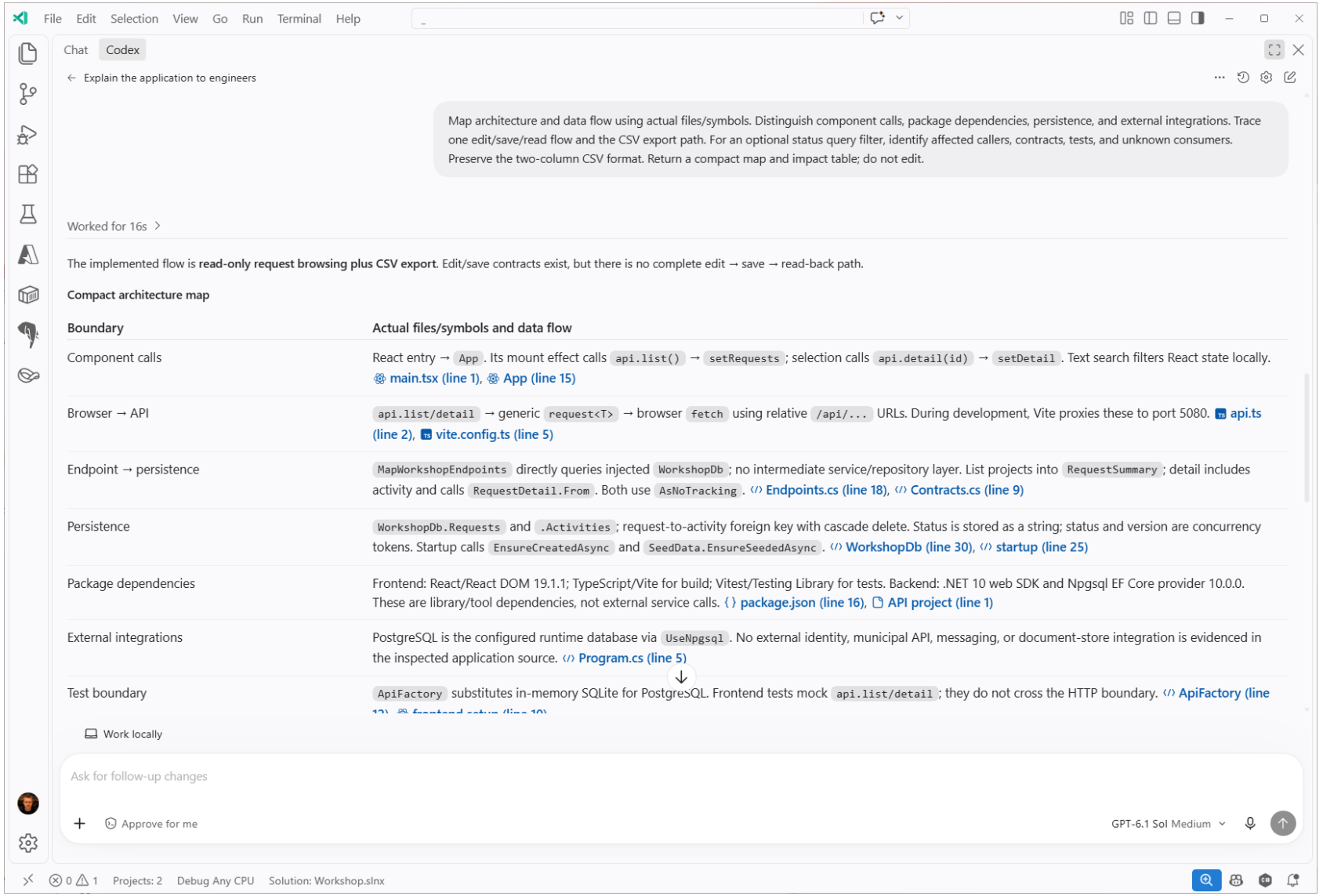
Turn this into **reusable knowledge**

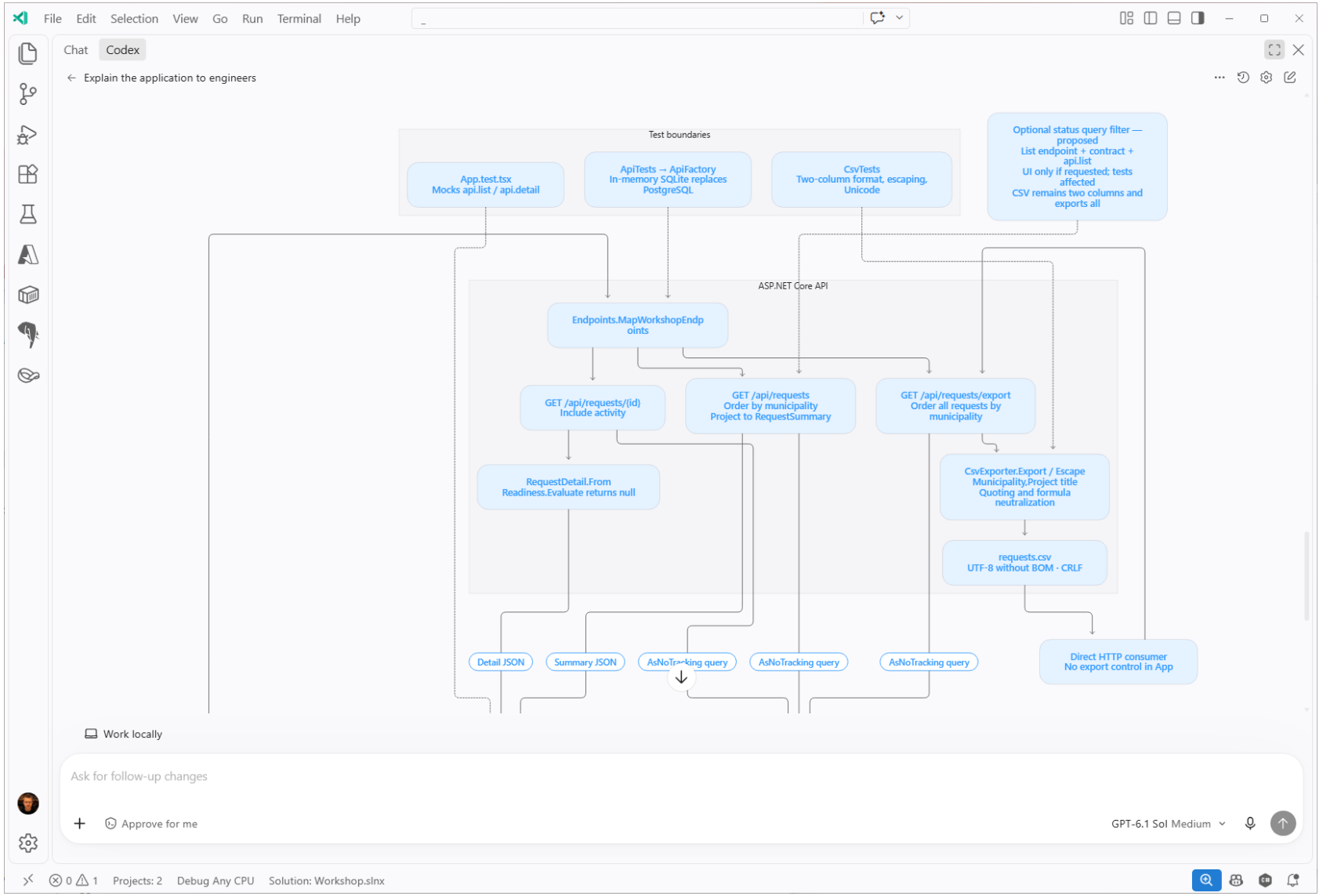
Discovery before development

demo


File Edit Selection View Go Run Terminal Help
Chat Codex
Explain the application to engineers
Explain this unfamiliar application to a new engineer. Identify users implied by the UI, main capabilities, entry points, important business processes, and external dependencies evidenced in the repository. Trace one list → details → edit → review journey where supported. Cite files/symbols. Mark unknowns. Do not edit and do not create or add fictional rules.
Worked for 33s
Request Workbench is a municipal project-request workspace. The current implementation supports browsing, searching, and reading requests, plus API-only CSV export. Editing, readiness checks, and marking requests Reviewed remain planned. This distinction is explicit in the API contract (line 7) and confirmed by Endpoints.MapWorkshopEndpoints, which registers only GET endpoints in Endpoints.cs (line 6).
Users and capabilities
The UI implies a caseworker or request officer handling municipal requests: it presents a request inbox, project details, status, and budget-presence information. Municipalities are data on requests; the code does not establish them as signed-in users. The contract describes one fictional development caseworker and no sign-in or permission system. See App (line 46), ActivityEntry.Actor (line 20), and the identity boundary (line 32).
Capability
Evidence and current behavior
Browse requests
api.list() loads municipality, project title, and status. The server orders by municipality. App (line 15), list endpoint (line 18)
Search
Filters already-loaded municipality and title values using trimmed, case-insensitive substring matching. Search makes no API calls; an open detail stays visible when filtered out. App (line 35)
Read details
Shows project title, municipality, description, budget flag, and status. detail panel (line 68)
Export
GET /api/requests/export exports all requests, ordered by municipality, with Municipality and Project title columns. No export control appears in App. endpoint (line 12), CsvExporter.Export (line 8)
Check service connectivity
GET /api/health calls the database connection check. health endpoint (line 8)
Entry points and architecture
The documented development entry is http://127.0.0.1:5173; Vite proxies /api to the ASP.NET Core service http://127.0.0.1:5080. README (line 21), Vite configuration (line 3).
The main code path is:
Work locally
Ask for follow-up changes
Approve for me
GPT-6.1 Sol Medium
0 1 Projects: 2 Debug Any CPU Solution: Workshop.slnx


File Edit Selection View Go Run Terminal Help
Chat
Codex
Explain the application to engineers
Map architecture and data flow using actual files/symbols. Distinguish component calls, package dependencies, persistence, and external integrations. Trace one edit/save/read flow and the CSV export path. For an optional status query filter, identify affected callers, contracts, tests, and unknown consumers. Preserve the two-column CSV format. Return a compact map and impact table; do not edit.
Worked for 16s
The implemented flow is read-only request browsing plus CSV export. Edit/save contracts exist, but there is no complete edit → save → read-back path.
Compact architecture map
Boundary
Actual files/symbols and data flow
Component calls
React entry → App. Its mount effect calls api.list() → setRequests; selection calls api.detail(id) → setDetail. Text search filters React state locally. main.tsx (line 1), App (line 15)
Browser → API
api.list/detail → generic request<T> → browser fetch using relative /api/... URLs. During development, Vite proxies these to port 5080. api.ts (line 2), vite.config.ts (line 5)
Endpoint → persistence
MapWorkshopEndpoints directly queries injected WorkshopDb; no intermediate service/repository layer. List projects into RequestSummary; detail includes activity and calls RequestDetail.From. Both use AsNoTracking. Endpoints.cs (line 18), Contracts.cs (line 9)
Persistence
WorkshopDb.Requests and .Activities; request-to-activity foreign key with cascade delete. Status is stored as a string; status and version are concurrency tokens. Startup calls EnsureCreatedAsync and SeedData.EnsureSeededAsync. WorkshopDb (line 30), startup (line 25)
Package dependencies
Frontend: React/React DOM 19.1.1; TypeScript/Vite for build; Vitest/Testing Library for tests. Backend: .NET 10 web SDK and Npgsql EF Core provider 10.0.0. These are library/tool dependencies, not external service calls. package.json (line 16), API project (line 1)
External integrations
PostgreSQL is the configured runtime database via UseNpgsql. No external identity, municipal API, messaging, or document-store integration is evidenced in the inspected application source. Program.cs (line 5)
Test boundary
ApiFactory substitutes in-memory SQLite for PostgreSQL. Frontend tests mock api.list/detail; they do not cross the HTTP boundary. ApiFactory (line
Work locally
Ask for follow-up changes
Approve for me
GPT-6.1 Sol Medium
Projects: 2
Debug Any CPU
Solution: Workshop.slnx


File Edit Selection View Go Run Terminal Help
Chat
Codex
Explain the application to engineers
Map architecture and data flow using actual files/symbols. Distinguish component calls, package dependencies, persistence, and external integrations. Trace one edit/save/read flow and the CSV export path. For an optional status query filter, identify affected callers, contracts, tests, and unknown consumers. Preserve the two-column CSV format. Return a compact map and impact table; do not edit.
Worked for 16s
The implemented flow is read-only request browsing plus CSV export. Edit/save contracts exist, but there is no complete edit → save → read-back path.
Compact architecture map
Boundary
Actual files/symbols and data flow
Component calls
React entry → App. Its mount effect calls api.list() → setRequests; selection calls api.detail(id) → setDetail. Text search filters React state locally. main.tsx (line 1), App (line 15)
Browser → API
api.list/detail → generic request<T> → browser fetch using relative /api/... URLs. During development, Vite proxies these to port 5080. api.ts (line 2), vite.config.ts (line 5)
Endpoint → persistence
MapWorkshopEndpoints directly queries injected WorkshopDb; no intermediate service/repository layer. List projects into RequestSummary; detail includes activity and calls RequestDetail.From. Both use AsNoTracking. Endpoints.cs (line 18), Contracts.cs (line 9)
Persistence
WorkshopDb.Requests and .Activities; request-to-activity foreign key with cascade delete. Status is stored as a string; status and version are concurrency tokens. Startup calls EnsureCreatedAsync and SeedData.EnsureSeededAsync. WorkshopDb (line 30), startup (line 25)
Package dependencies
Frontend: React/React DOM 19.1.1; TypeScript/Vite for build; Vitest/Testing Library for tests. Backend: .NET 10 web SDK and Npgsql EF Core provider 10.0.0. These are library/tool dependencies, not external service calls. package.json (line 16), API project (line 1)
External integrations
PostgreSQL is the configured runtime database via UseNpgsql. No external identity, municipal API, messaging, or document-store integration is evidenced in the inspected application source. Program.cs (line 5)
Test boundary
ApiFactory substitutes in-memory SQLite for PostgreSQL. Frontend tests mock api.list/detail; they do not cross the HTTP boundary. ApiFactory (line
Work locally
Ask for follow-up changes
Approve for me
GPT-6.1 Sol Medium
Projects: 2
Debug Any CPU
Solution: Workshop.slnx


File Edit Selection View Go Run Terminal Help
Chat
Codex
Explain the application to engineers
Test boundaries
App.test.tsx
Mocks api.list / api.detail
ApiTests → ApiFactory
In-memory SQLite replaces PostgreSQL
CsvTests
Two-column format, escaping, Unicode
Optional status query filter — proposed
List endpoint + contract + api.list
UI only if requested; tests affected
CSV remains two columns and exports all
ASP.NET Core API
Endpoints.MapWorkshopEndpoints
GET /api/requests/{id}
Include activity
GET /api/requests
Order by municipality
Project to RequestSummary
GET /api/requests/export
Order all requests by municipality
RequestDetail.From
Readiness.Evaluate returns null
CsvExporter.Export / Escape
Municipality,Project title
Quoting and formula neutralization
requests.csv
UTF-8 without BOM · CRLF
Detail JSON
Summary JSON
AsNoTracking query
AsNoTracking query
AsNoTracking query
Direct HTTP consumer
No export control in App
Work locally
Ask for follow-up changes
Approve for me
GPT-6.1 Sol Medium
Projects: 2
Debug Any CPU
Solution: Workshop.slnx

Reusable knowledge

Instructions

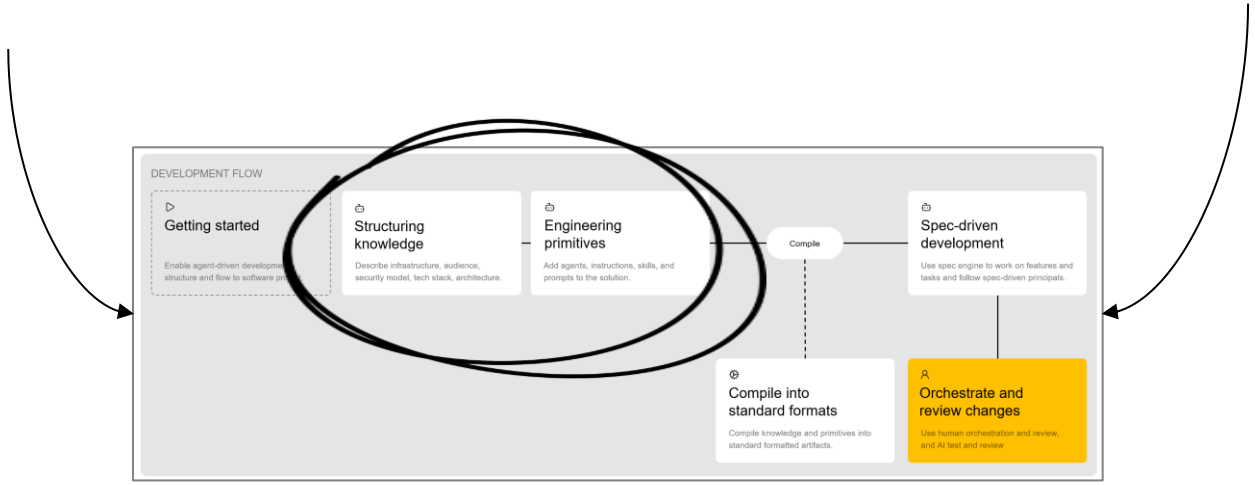
- **AGENTS.md**
  project instructions, architectural boundaries, development conventions, and verification commands.
- **Specialized agents**
  for architecture reviews, security reviews, and domain-specific validation.

Knowledge

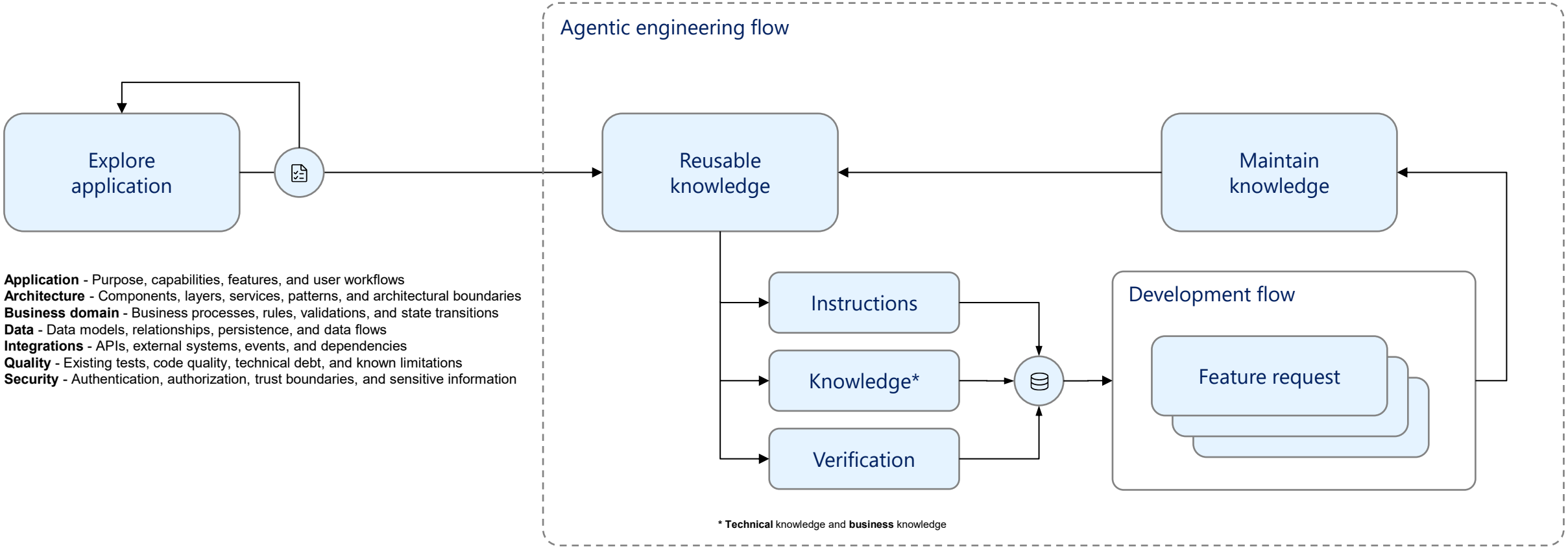
- **Architecture documentation**
  components, dependencies, integrations, and important design decisions.
- **Business rule documentation**
  rules discovered in the implementation and subsequently confirmed by domain experts.

Verification

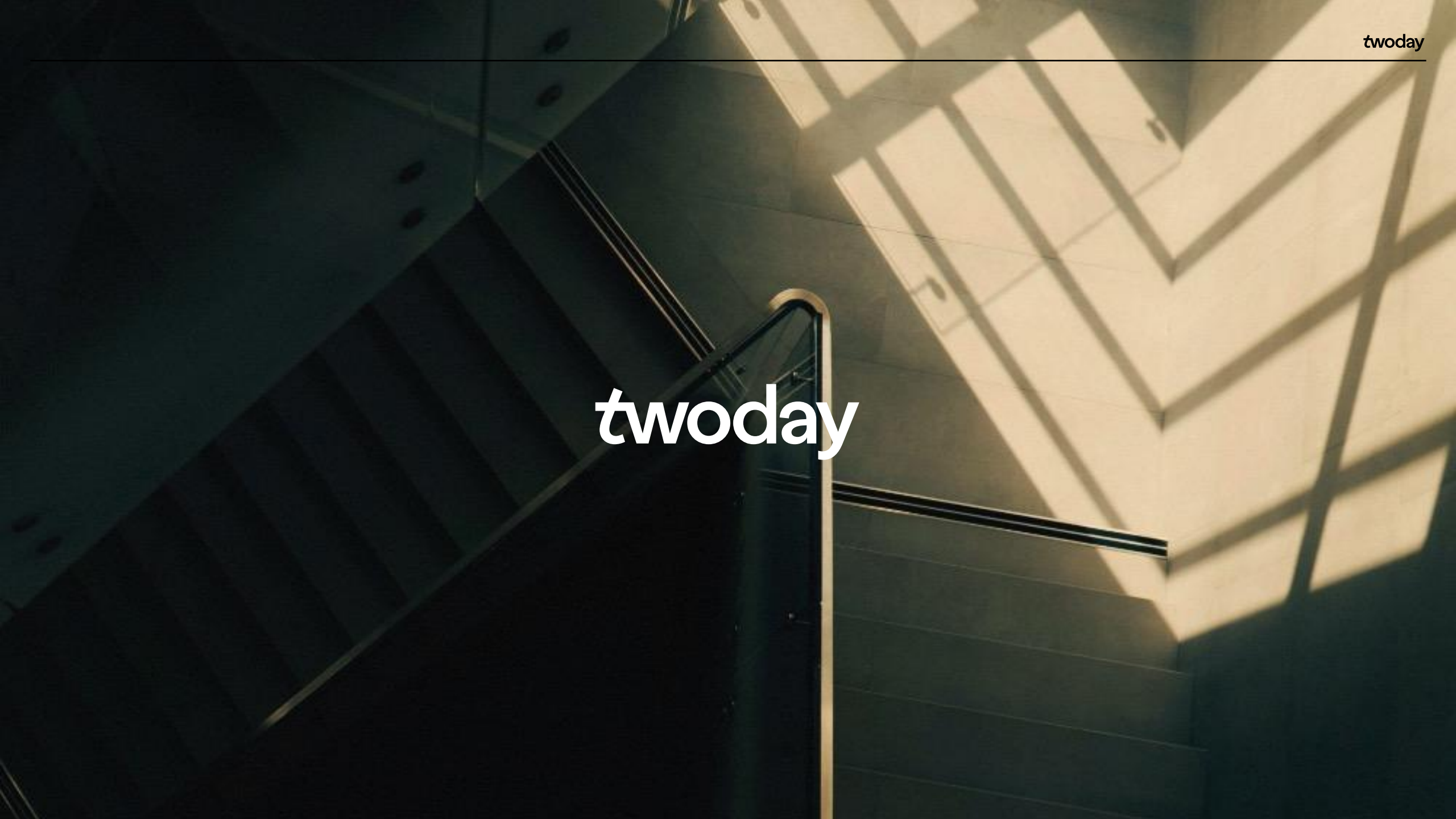
- **Tests**
  executable evidence of expected behavior and important existing contracts.



Continuously maintain project knowledge



Application - Purpose, capabilities, features, and user workflows
Architecture - Components, layers, services, patterns, and architectural boundaries
Business domain - Business processes, rules, validations, and state transitions
Data - Data models, relationships, persistence, and data flows
Integrations - APIs, external systems, events, and dependencies
Quality - Existing tests, code quality, technical debt, and known limitations
Security - Authentication, authorization, trust boundaries, and sensitive information


twoday